Pipelining = an implementation technique in which multiple instr. are overlapped in execution.

4 tasks

Washer / Dryer / folder / stower

4 loads — 30 minutes

Sequential Laundry

$= (0.5 \times 4) \times 4 = 8$ hours

$$\text{Speedup (n loads)} = \frac{(4 \times 0.5) \times n}{(4 \times 0.5) + (n-1) \times 0.5}$$

$$= \frac{2n}{2 + (n-1) \times 0.5}$$

$$\approx \frac{2n}{(n-1) \times 0.5}$$

$$\approx 4$$

pipelining increase Instr. throughput

It does not decrease execution time of individual instr.

Pipeline stage times are limited by the slowest resource of the Computer.



Structural Hazard = when a planned instruction can not execute in the proper cycle because hardware does not support the combination of instructions that are set to execute.

* Suppose we have single memory instead of two memories (Instr Memory & Data Memory)

	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8
Instr 1	IF	ID	EX	MEM	WB			
Instr 2		IF	ID	EX	MEM	WB		
Instr 3			IF	ID	EX	MEM	WB	
Instr 4				IF	ID	EX	MEM	WB

In C_4 → $Instr_1$ access data from memory & $Instr_4$ fetch instr from memory

⇒ STRUCTURAL HAZARD

Seperate memories ⇒ NO STRUCTURAL HAZARD

Data Hazard = When a planned instr. can not execute in the proper clock cycle because data that is needed to execute the instruction is not yet available.

Pipeline Stall / Bubble = for resolving a hazard

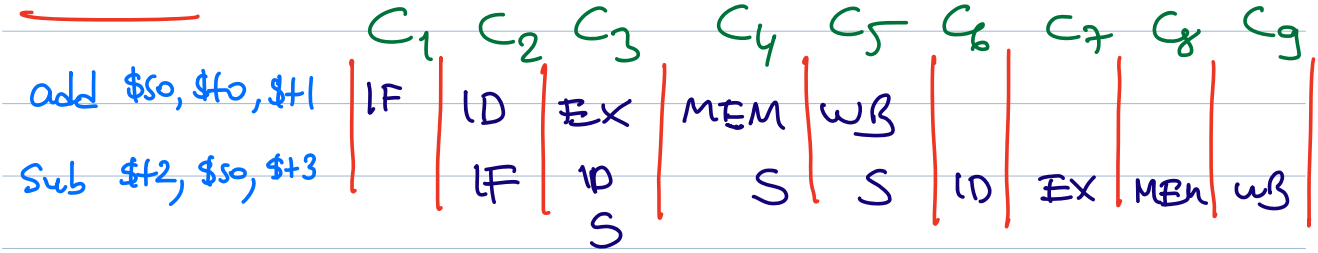
	C_1	C_2	C_3	C_4	C_5	C_6
• add $s0, $t0, $t1	IF	ID	EX.	MEM	WB	
• sub $t2, $s0, $t3		IF	ID	EX	MEM	WB

* Shading = elements that is used by the instr.

Shading on right half of register file/memory
⇒ READ

Shading on left half of register file/memory
⇒ WRITE

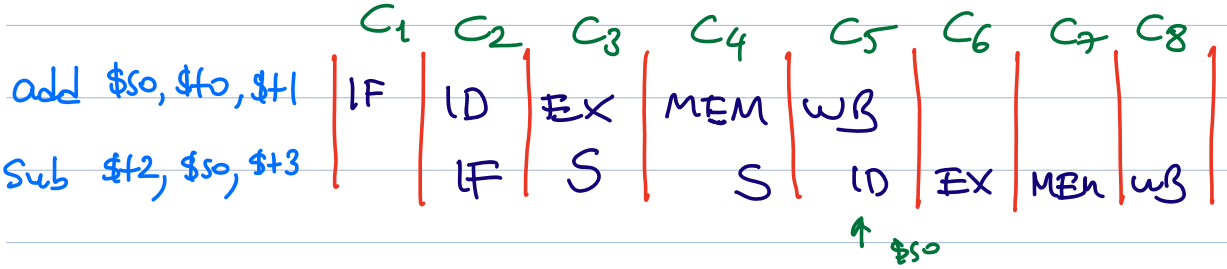
CASE 1



We waste 3 cycles # of CC = 9

CASE 2 :

Assumption = Write register file in first half of CC & read register file in second half of CC.

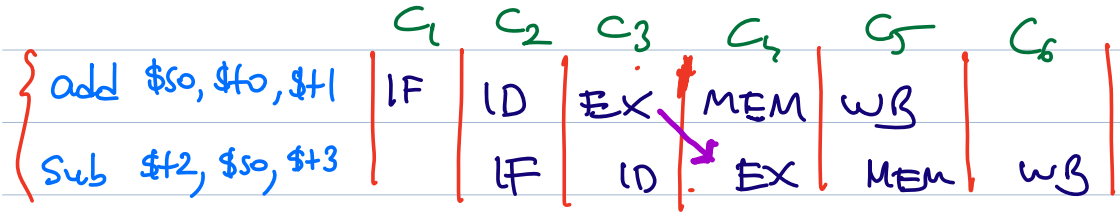


We waste 2 cycles # of CC = 8

CASE 3

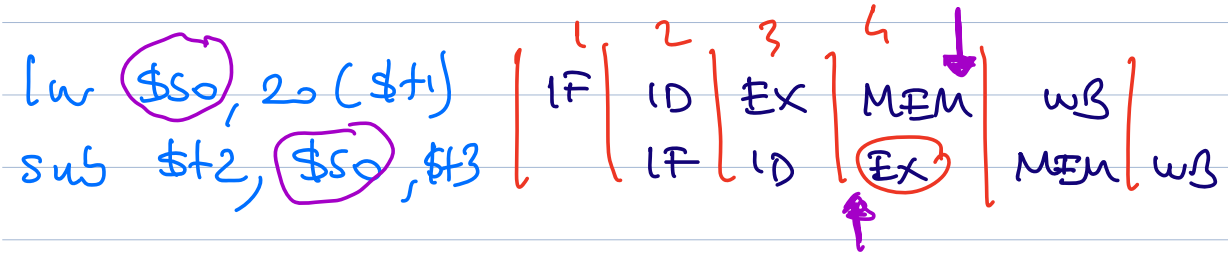
No need to wait for instr. to complete before trying to resolve the data hazard.

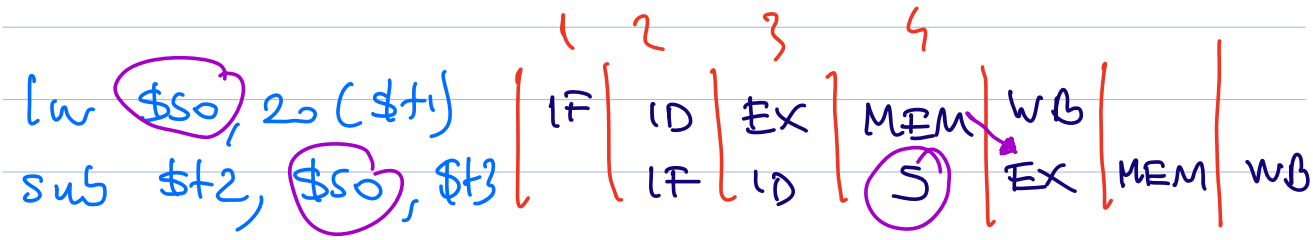
(Forwarding / Bypassing = Retrieve missing data element from internal buffers rather than wait for them.

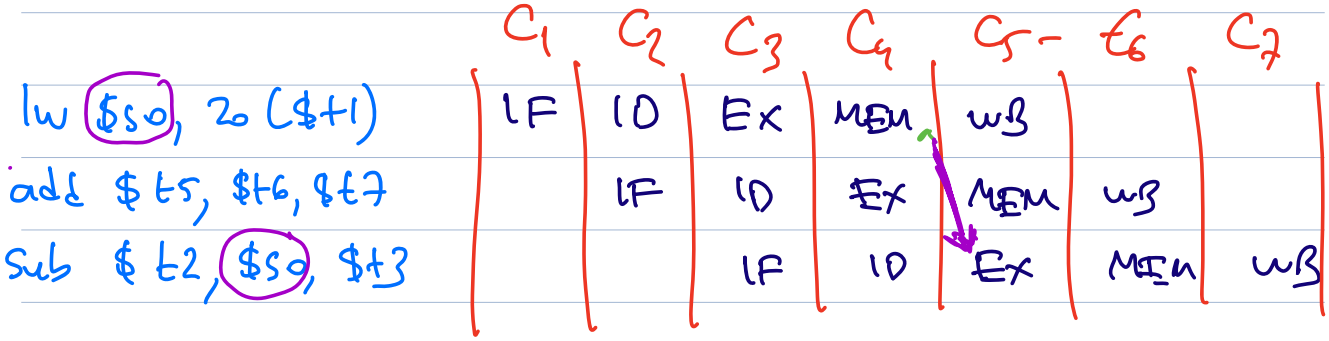


of clock cycles = 6

Load-use Data Hazard





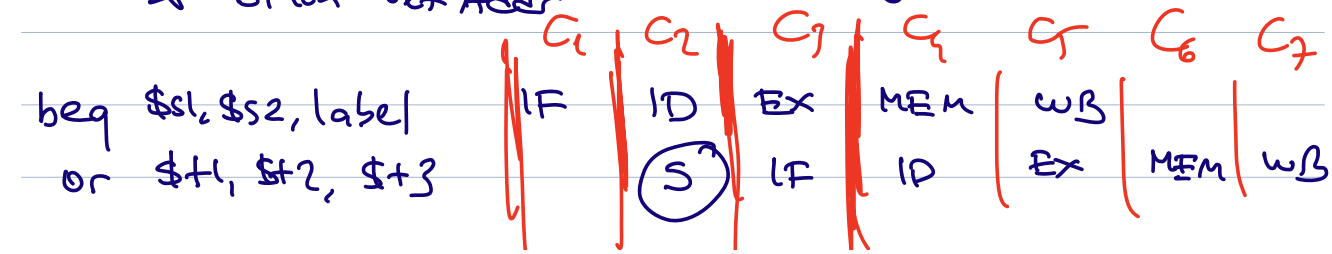


Control Hazards = When the proper instr. cannot execute in the proper pipeline clock cycle because the instruction that was feteked is not the one that is needed

Solution 1: Wait until branch outcome determined before fetching the next instruction.

Assumption: We put extra hardware so that we can test registers calculate branch address & update PC during the second stage.

Branch Decision & Branch Target Addr → ID stage



(If assumption not applied)

beq $s1, $s2, label
or $t1, $t2, $t3



Solution 2: Predict that branch will not be taken

Solution 3: Predict that branch will be taken

*** Study the examples given in the textbook.